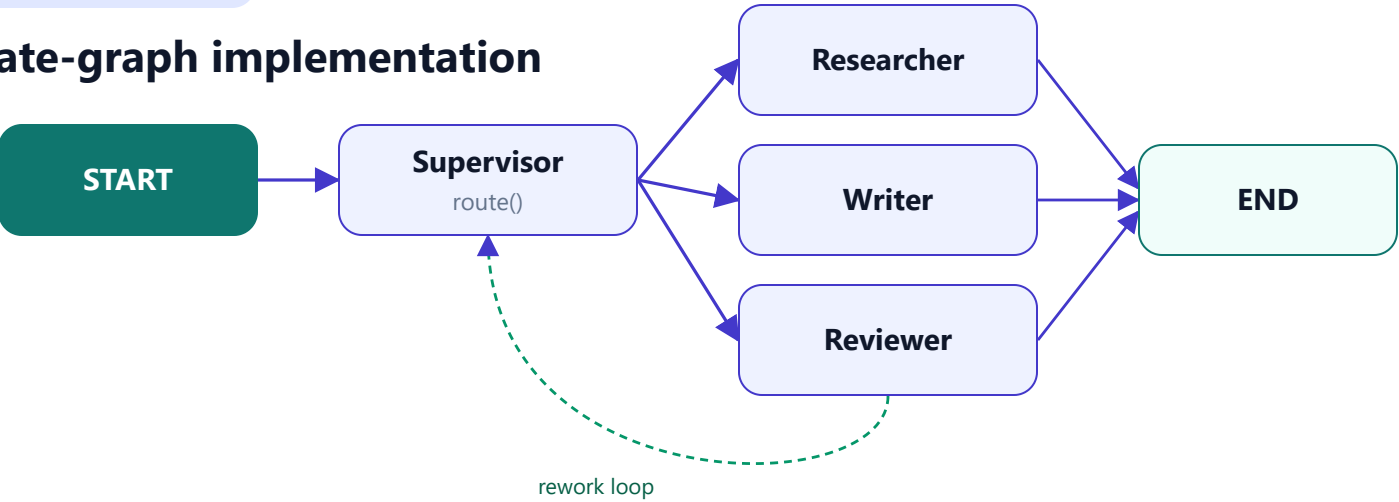


LangGraph & Microsoft Agent Framework Lab

Hands-on: build the same 3-agent workflow twice — once as a LangGraph state machine, once as a MAF orchestration — and compare.

LAB A · LANGGRAPH

State-graph implementation



```
# build_graph.py
from langgraph.graph import StateGraph, END
from agents import researcher, writer, reviewer, route

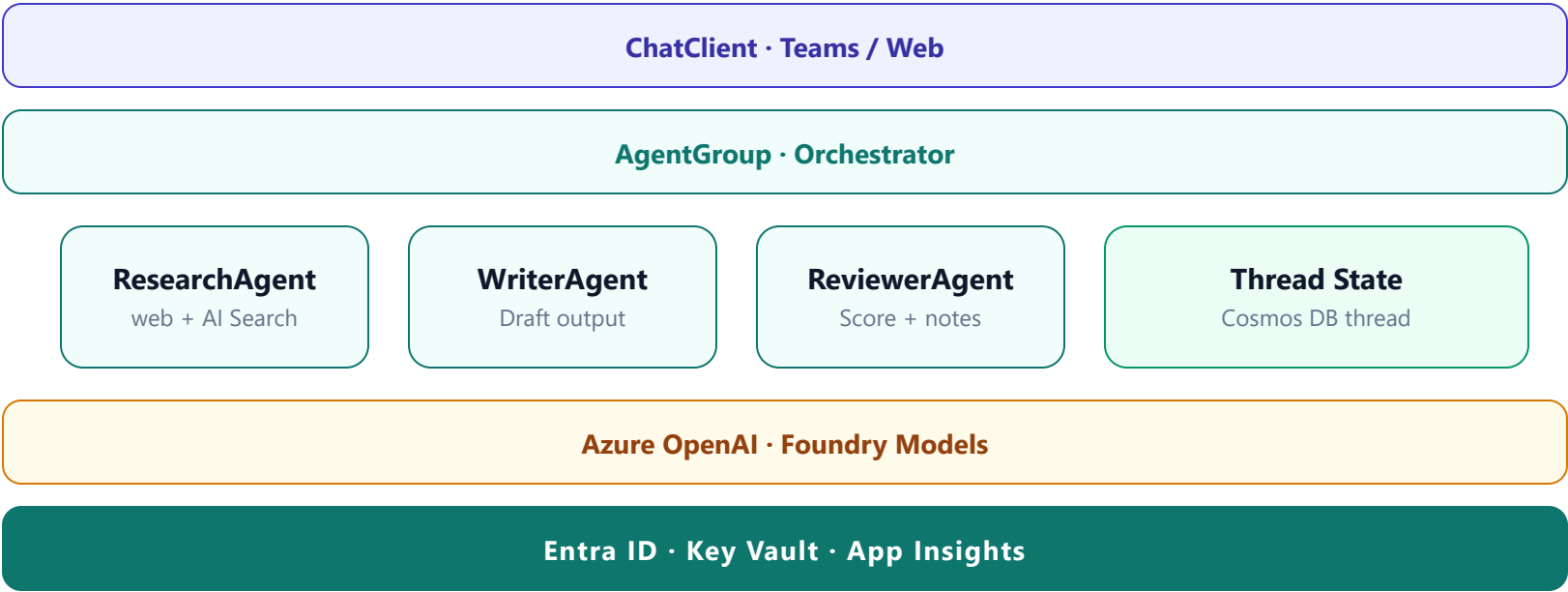
graph = StateGraph(AgentState)
graph.add_node("researcher", researcher)
graph.add_node("writer", writer)
graph.add_node("reviewer", reviewer)
graph.add_conditional_edges("reviewer", route,
    {"rework": "writer", "done": END})
app = graph.compile(checkpointer=SqliteSaver.from_conn_string(":memory:"))
```

Lab exercises · 90 min

- 1 · Define AgentState TypedDict & add reducers
- 2 · Wire conditional edge on quality score threshold
- 3 · Enable checkpointing; replay from step 2
- 4 · Stream token events to a Gradio UI
- 5 · Insert HITL interrupt before writer publishes

LAB B · MICROSOFT AGENT FRAMEWORK

Same workflow · Azure-native runtime



```
// Program.cs · .NET 9
var chat = new AzureOpenAIClient(endpoint, cred);
var researcher = new ChatCompletionAgent(chat, tools: [webSearch]);
var writer = new ChatCompletionAgent(chat);
var reviewer = new ChatCompletionAgent(chat);

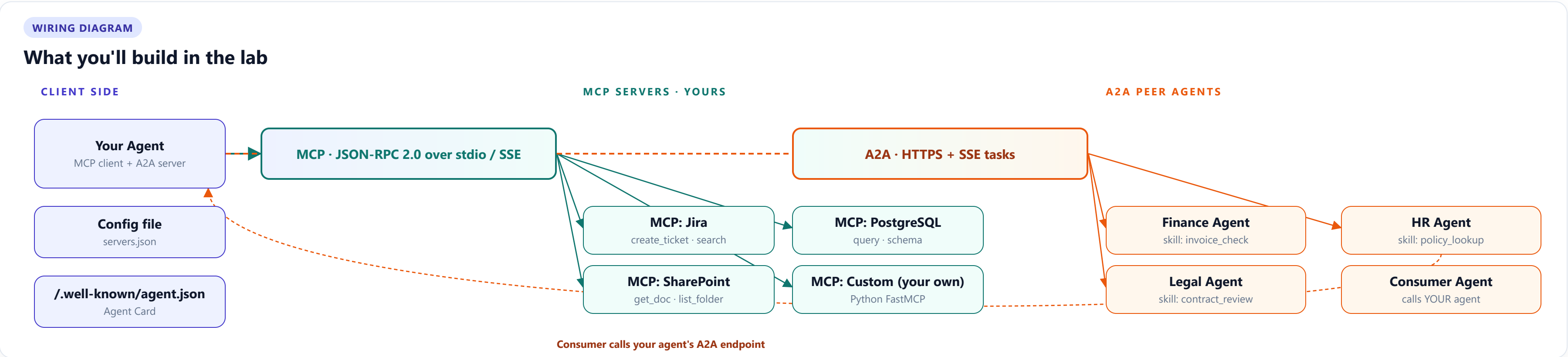
var group = new AgentGroupChat(researcher, writer, reviewer)
    { TerminationStrategy = new ApprovalTermination(reviewer) };
await foreach (var msg in group.InvokeStreamingAsync(input))
    Console.Write(msg.Content);
```

Compare & report

Deliverables: latency + token graphs · developer-experience notes · a written verdict on which runtime you'd pick for a regulated .NET shop vs. a Python data-science team.

MCP & A2A Integration Lab

Ship a custom MCP server, connect it to your agent, then expose your agent to peers via A2A. End-to-end wiring.



TASK 1 · BUILD A CUSTOM MCP SERVER

Expose 2 tools + 1 resource

```
# server.py · FastMCP
from mcp.server.fastmcp import FastMCP
mcp = FastMCP("orders-server")

@mcp.tool()
def get_order(order_id: str) -> dict:
    return db.fetch(order_id)

@mcp.resource("orders://recent/{n}")
def recent_orders(n: int) -> str: ...

mcp.run(transport="sse", port=8080)
```

TASK 2 · PUBLISH YOUR AGENT VIA A2A

Discover + delegate a task

1

 Ship Agent Card at /.well-known/agent.json — name, skills, auth.

2

 Wire OAuth2 / Entra token acceptance on /tasks.

3

 Stream SSE lifecycle: submitted → working → completed.

4

 Return artifacts (JSON + Markdown) with source citations.

5

 From your agent, call Finance peer to delegate an invoice check.

6

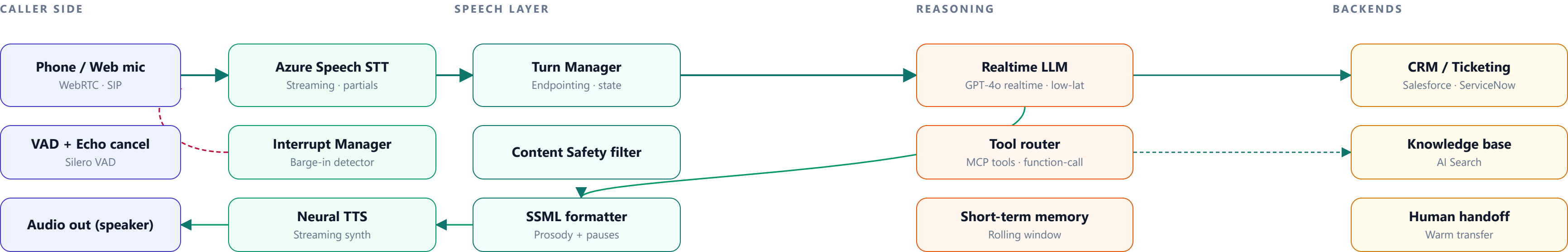
 Audit — persist task-id + full transcript to Cosmos.

Voice Agent Lab

Build a real-time voice agent — stream audio in, reason with tools, stream audio out — with barge-in and sub-second latency.

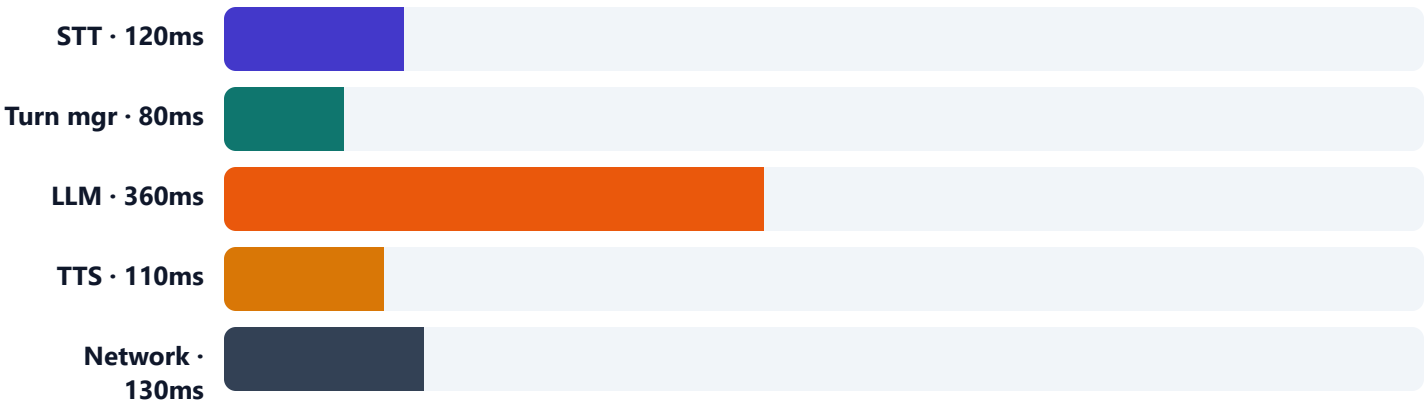
FLOW DIAGRAM

Real-time voice pipeline · barge-in enabled



LATENCY BUDGET · < 800 MS PERCEIVED

Where the milliseconds go



Streaming TTS starts as soon as the first LLM token arrives — perceived latency ≈ time-to-first-audio, not total pipeline time.

LAB EXERCISES

Ship a voice agent that books an appointment

- 1 · Stand up STT → LLM → TTS with WebRTC front-end
- 2 · Add barge-in: cancel TTS when caller starts speaking mid-response
- 3 · Give the LLM 3 tools: `list_slots` · `book_slot` · `send_confirmation`
- 4 · Add SSML prosody + strategic pauses for natural cadence
- 5 · Implement graceful **warm transfer** to human when confidence drops
- 6 · Record + play back your own conversation; measure round-trip latency

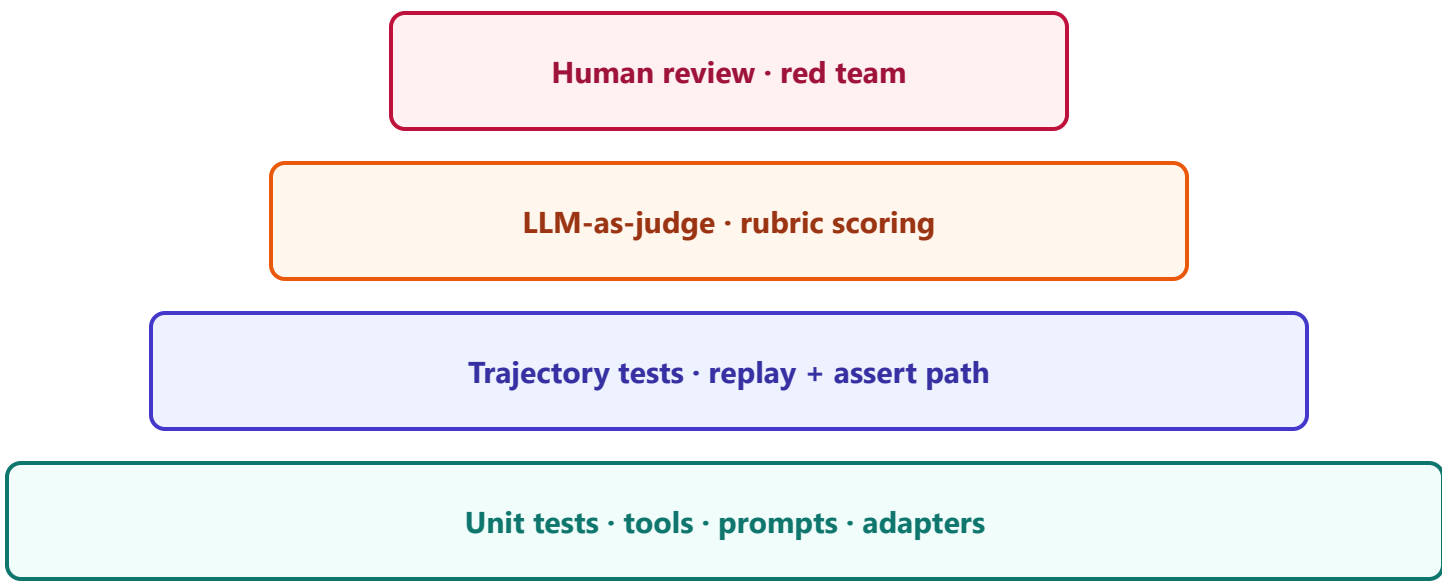
Deliverable: a 60-second demo call + a latency waterfall from the browser DevTools trace.

Testing & Evaluating Agents

A three-layer test pyramid — unit tests for tools, trajectory tests for reasoning, LLM-as-judge for open-ended quality.

TEST PYRAMID

Four layers, thickest at the base



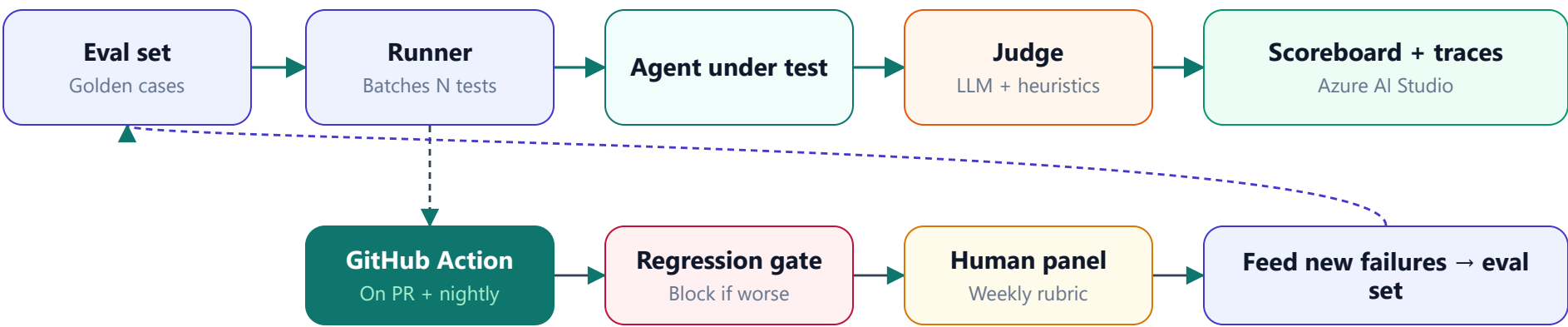
- HUMAN REVIEW** Curated eval set · pre-release gate · < 100 cases · slow, expensive, authoritative.
- LLM-AS-JUDGE** Rubrics for helpfulness · faithfulness · tone · safety. Correlate against human panel weekly.
- TRAJECTORY** Given input X, agent must call tools [a, b, c] in order and land at final state Y.
- UNIT** Every MCP tool has pytest cases. Prompts have snapshot tests. Fastest feedback loop.

Metrics that matter

- Task success rate** — end-to-end completion %
- Tool-call precision** — right tool, right args
- Cost per resolution** — tokens × price / task
- Regression delta** — vs. last known good build
- Groundedness** — % claims traceable to sources
- Latency P50 / P95** — time-to-first-token & total
- Escalation rate** — % routed to human
- Refusal correctness** — refused when it should

EVAL PIPELINE

CI-driven evaluation loop



Every failure the agent commits in production is turned into a golden test case the next day. The eval set grows with the product.

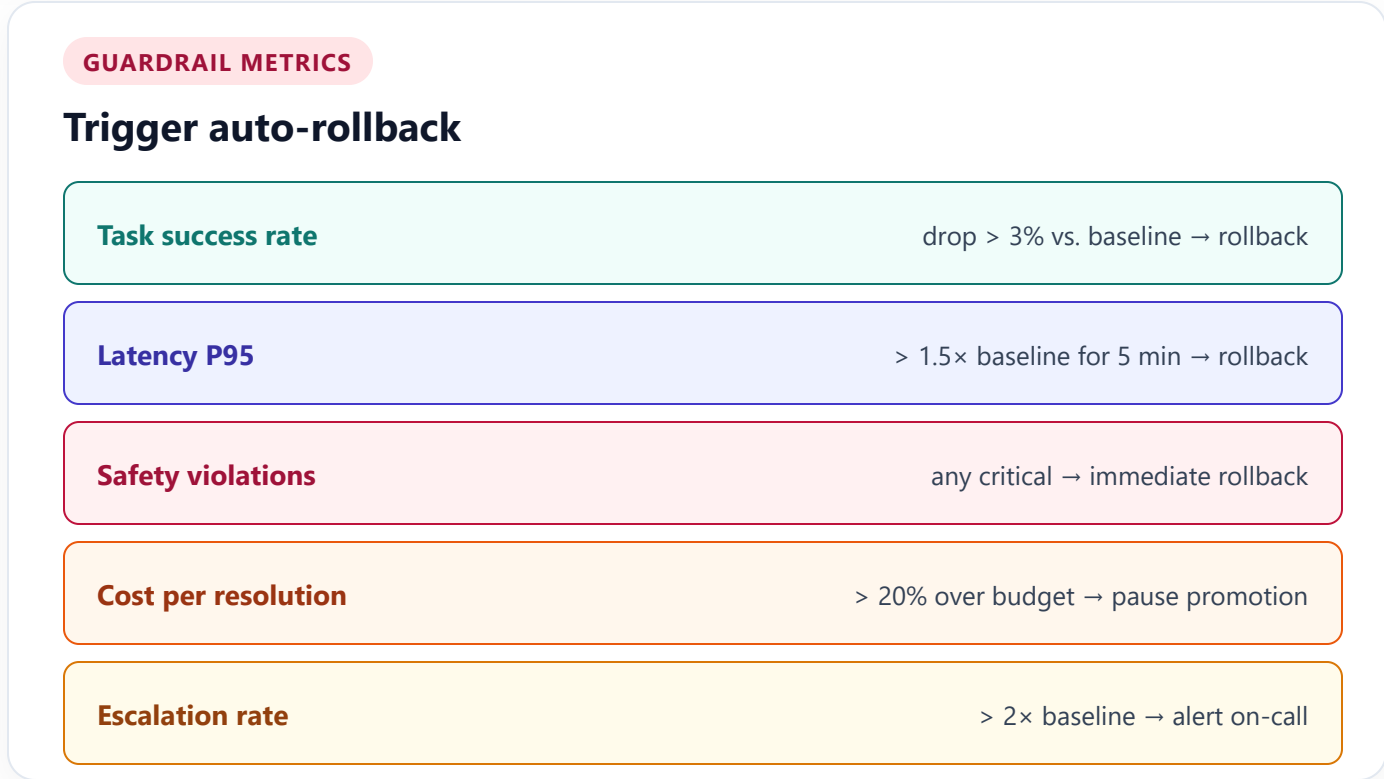
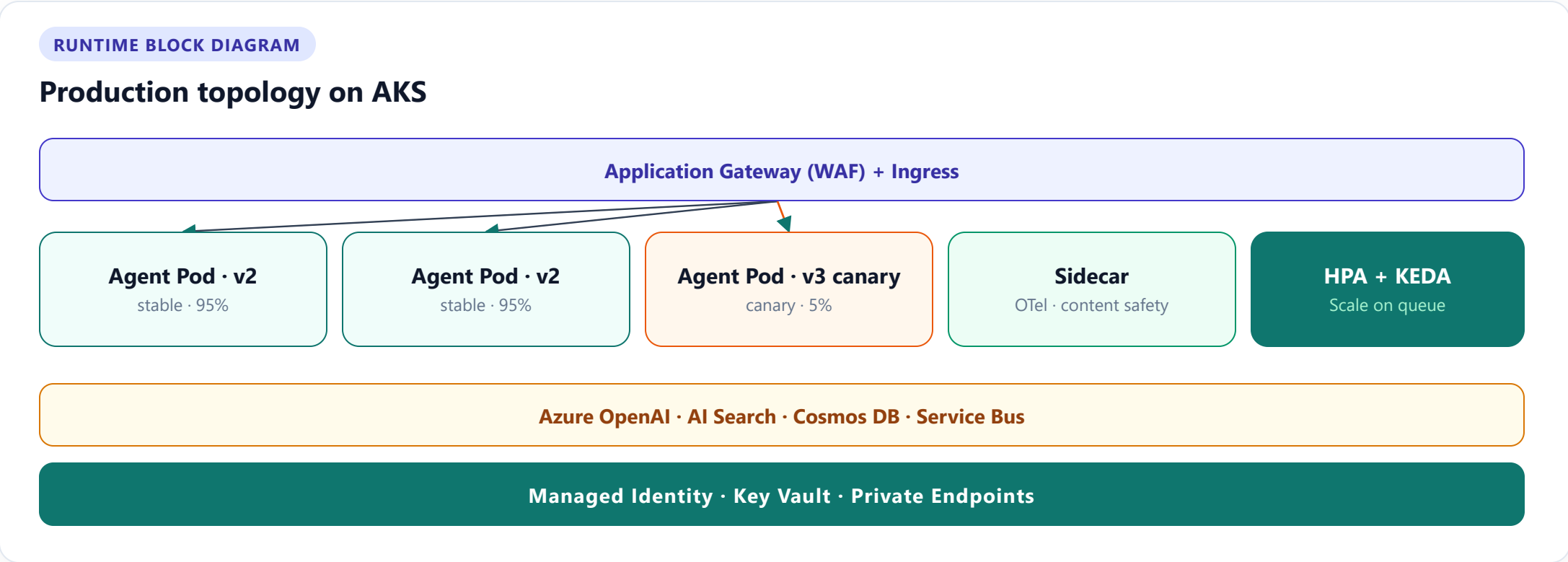
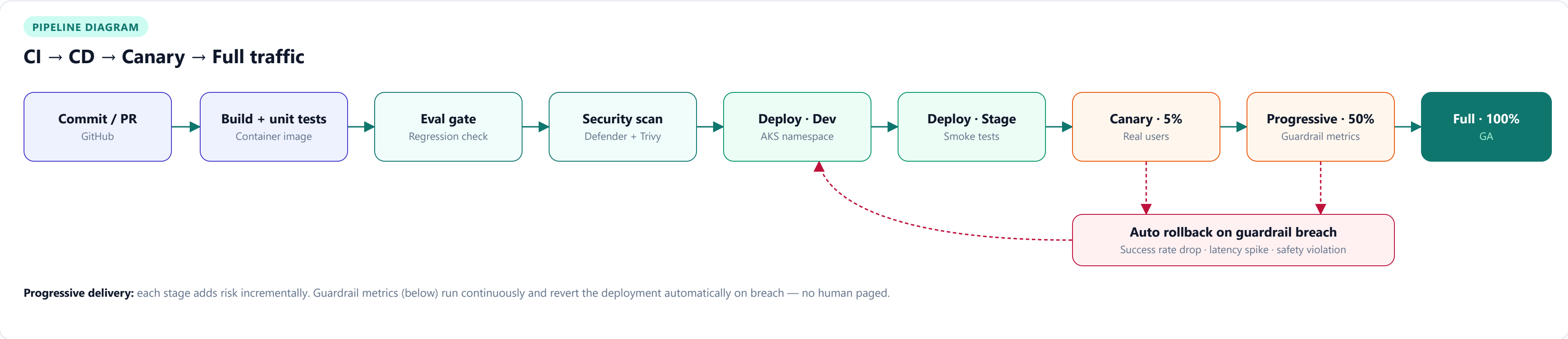
LLM-AS-JUDGE · RUBRIC EXAMPLE

Faithfulness · 5-point scale

Score	Definition	Judge decision cue
5	Every claim cites a retrieved span	All spans verified, no invented facts
4	Minor unsupported paraphrase	Gist correct, phrasing loose
3	One factual claim unsupported	Would confuse a novice reader
2	Multiple hallucinated details	Sources contradict output
1	Answer fabricated entirely	No overlap with retrieved evidence

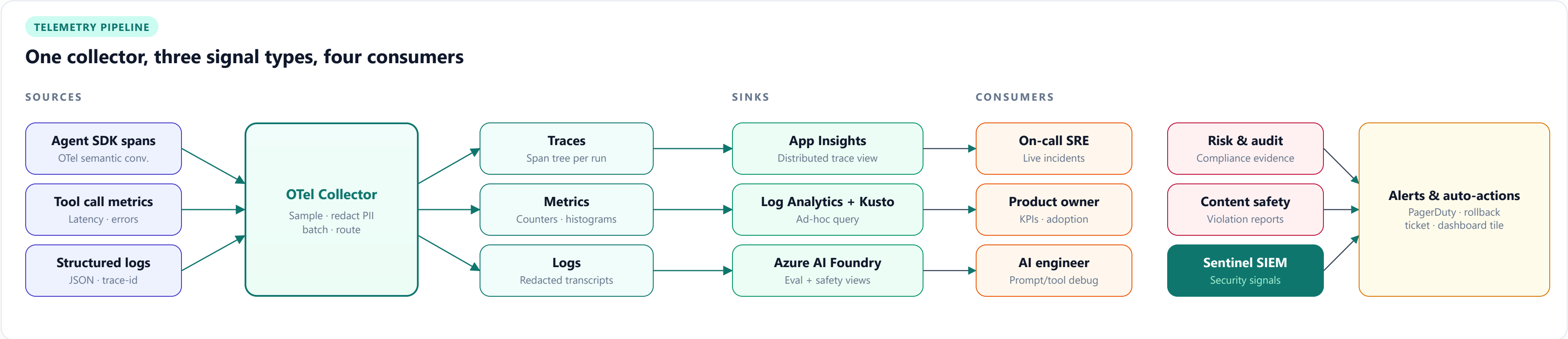
Deploying & Operating Agents

From git push to production traffic — with promotion gates, canaries, and safe rollback.



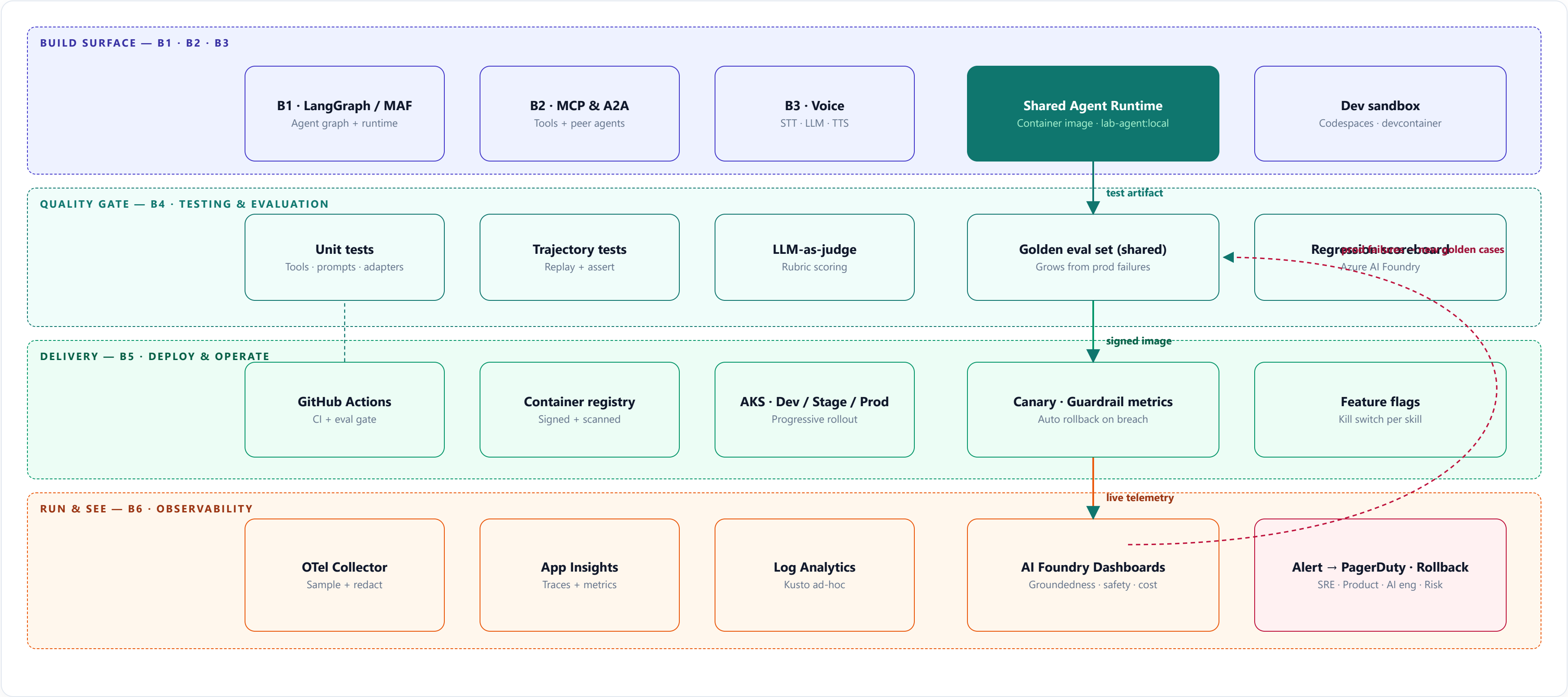
Observability Deep Dive

Traces, metrics, and structured logs for agents — one OpenTelemetry pipeline, four operator personas.



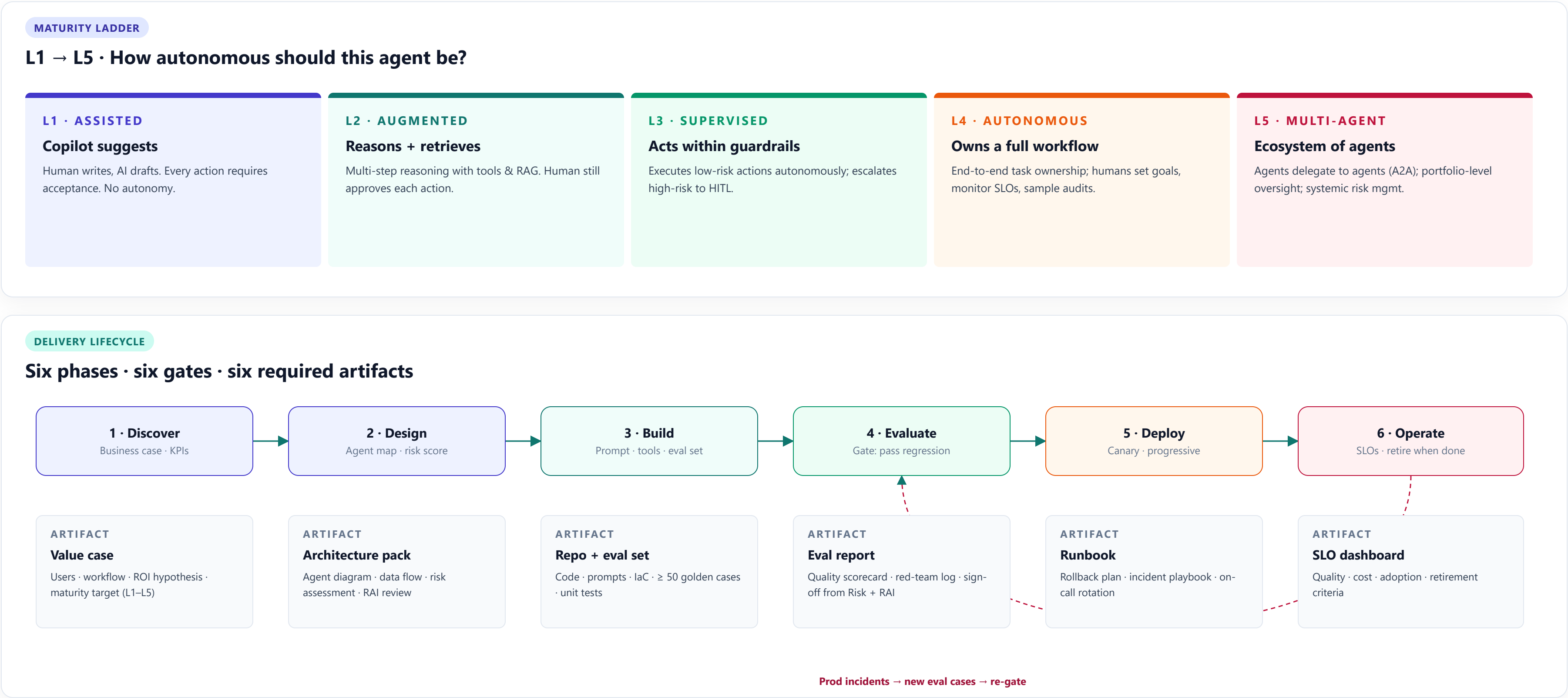
The Build-Lab Bench — How B1–B6 Wire Together

One environment, six labs. Every module writes into the same telemetry, feeds the same eval set, and ships through the same pipeline.



Agent Maturity & Delivery Lifecycle

Five maturity levels and the six lifecycle phases every agent product travels through — with the artifacts required at each gate.



Risk in Agentic Systems

Six failure modes unique to autonomous agents — with controls mapped to each. Score every use case before build.

FAILURE MODES

The six risks that agentic systems introduce

R1 · HALLUCINATION

Invented facts

Confident false statements. Users can't tell without checking.

Controls: RAG · groundedness score · refuse-without-source

R2 · PROMPT INJECTION

Adversarial instructions

Retrieved doc or user input hijacks the agent's behaviour.

Controls: content-role tagging · sandboxed tool args · allowlist

R3 · EXCESSIVE AGENCY

Wrong action, right intent

Agent takes an irreversible action that a human would have caught.

Controls: HITL on writes · dry-run · scoped credentials

R4 · DATA LEAKAGE

Cross-tenant / PII spill

Agent surfaces data the user isn't entitled to see, or sends PII outbound.

Controls: ACL-aware retrieval · PII redaction · egress DLP

R5 · BIAS & FAIRNESS

Unequal outcomes

Systemic disadvantage to a protected group across many decisions.

Controls: subgroup eval · fairness metrics · policy overrides

R6 · COST & DOS

Runaway loops

Infinite ReAct loops, exhausted rate limits, unbounded tool fanout.

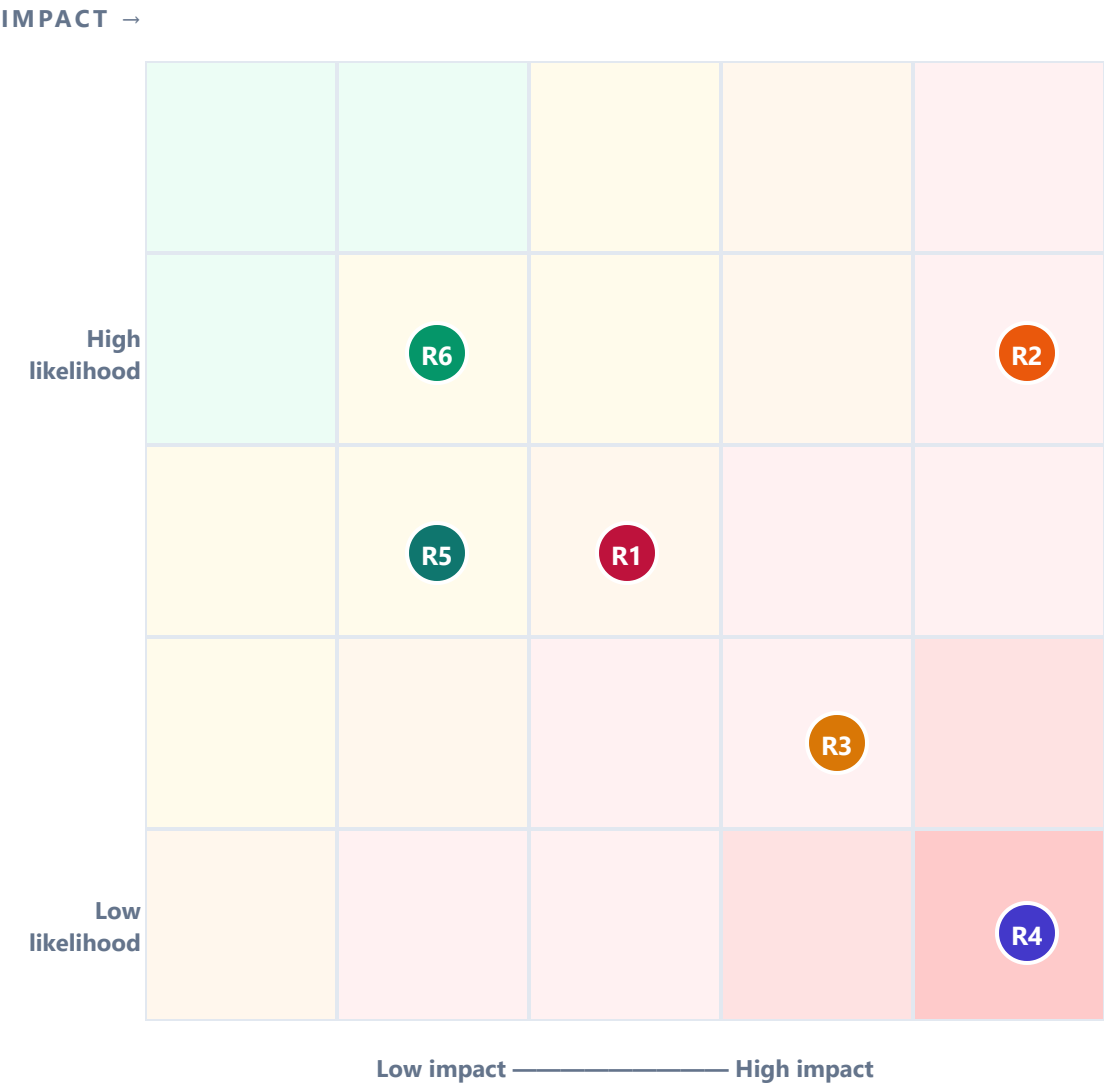
Controls: max-steps · token cap · circuit breakers

Every use case gets scored on all six axes — 1 (low) to 5 (high). Total score determines maturity ceiling: score > 20 caps the agent at L2 until controls are proven.

Framework aligns with NIST AI RMF (Govern / Map / Measure / Manage) and the EU AI Act's high-risk system categories.

RISK HEATMAP

Likelihood × Impact

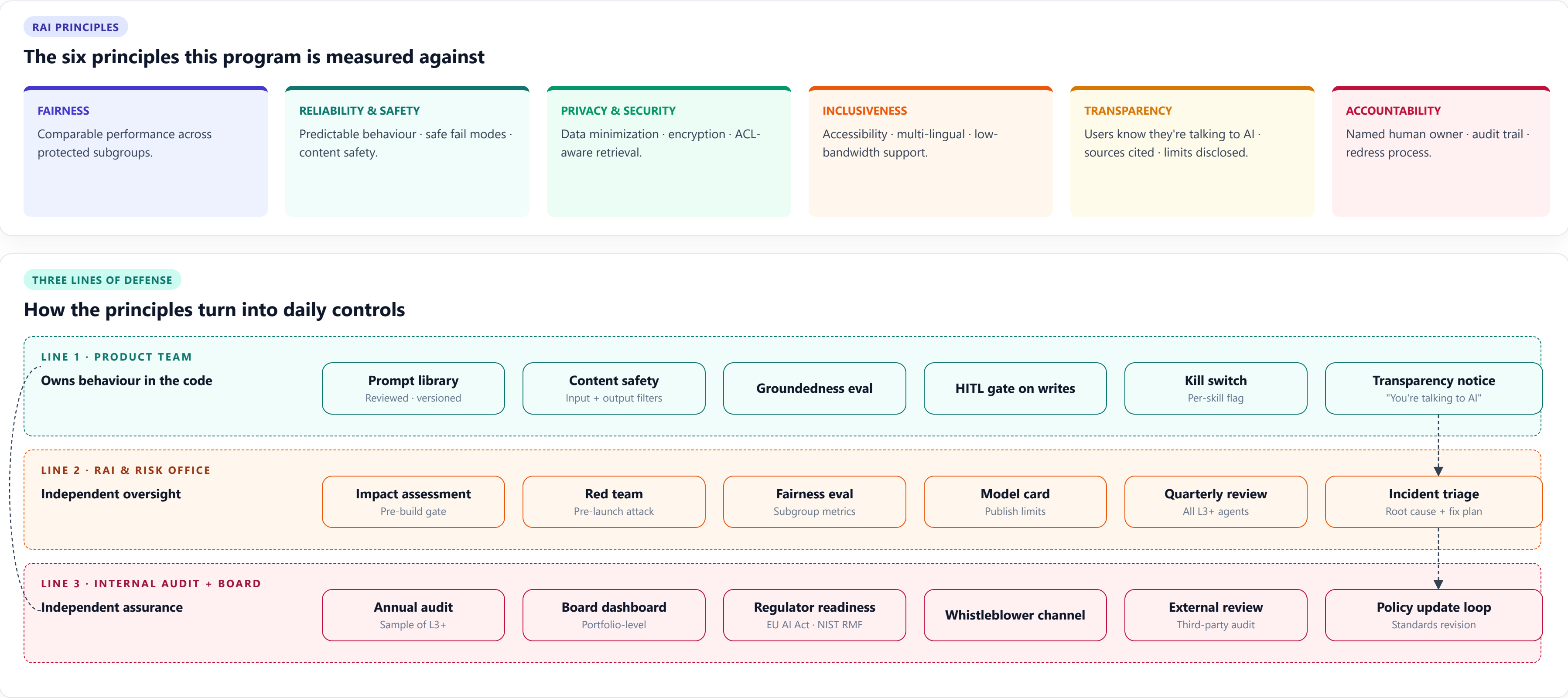


Red zone: requires L2 ceiling, HITL on every write, quarterly RAI review, and Board-visible dashboard.

Green zone: may proceed to L4 with standard controls.

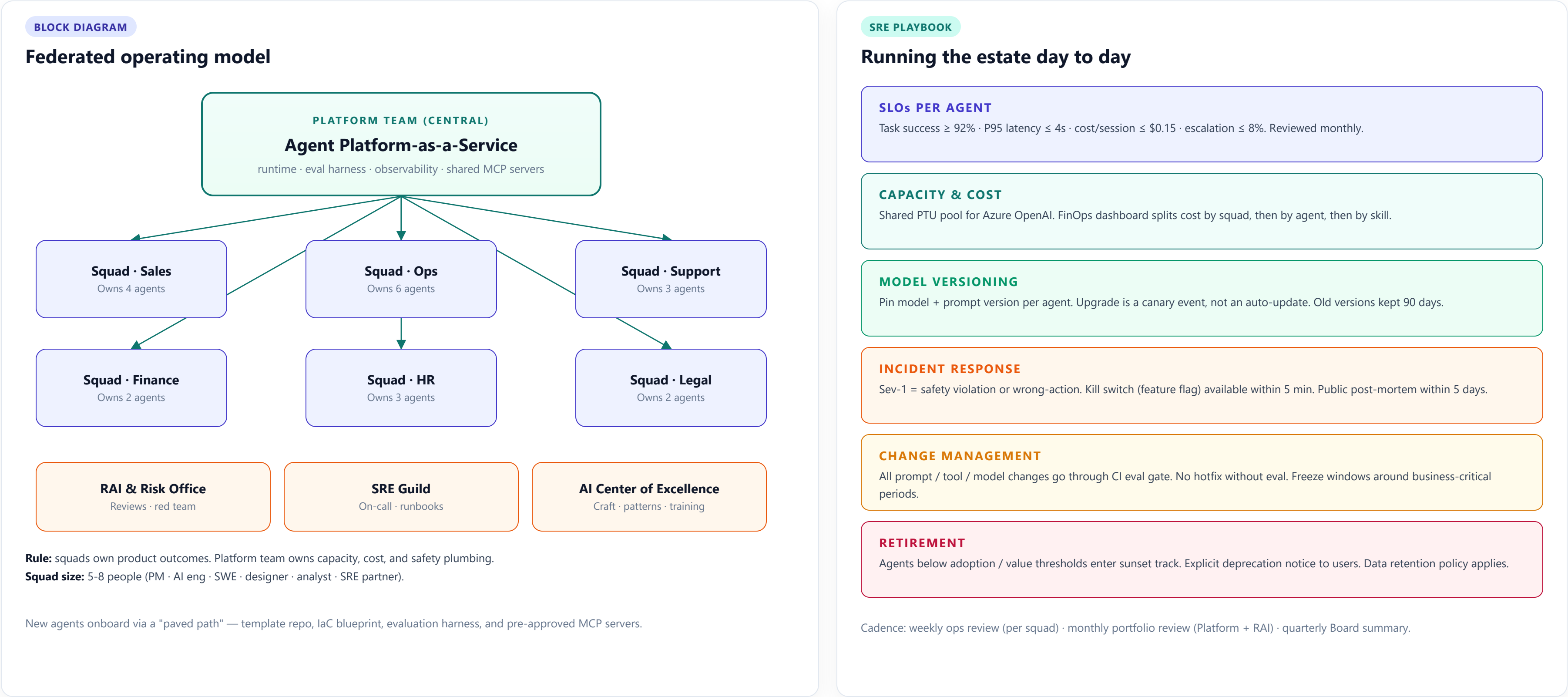
Responsible AI & Governance Deep Dive

Six RAI principles enforced through three lines of defense — with the concrete control that lives at each layer.



Operating Agents at Scale

From 1 agent to 50. The operating model, the SRE playbook, and the shared platform that makes multi-agent estates tractable.

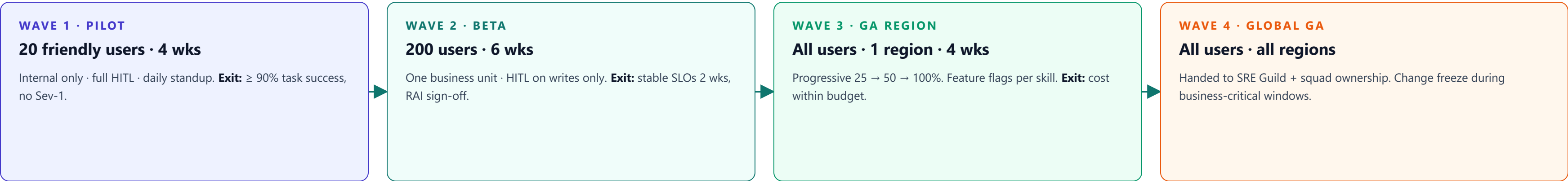


Rollout, RAID & Delivery Tabletop

A four-wave rollout plan and a live RAID log — Risks, Assumptions, Issues, Dependencies — updated every stand-up.

ROLLOUT WAVES · FLOW DIAGRAM

Four waves · exit criteria at each



RAID LOG · TABLETOP EXAMPLE

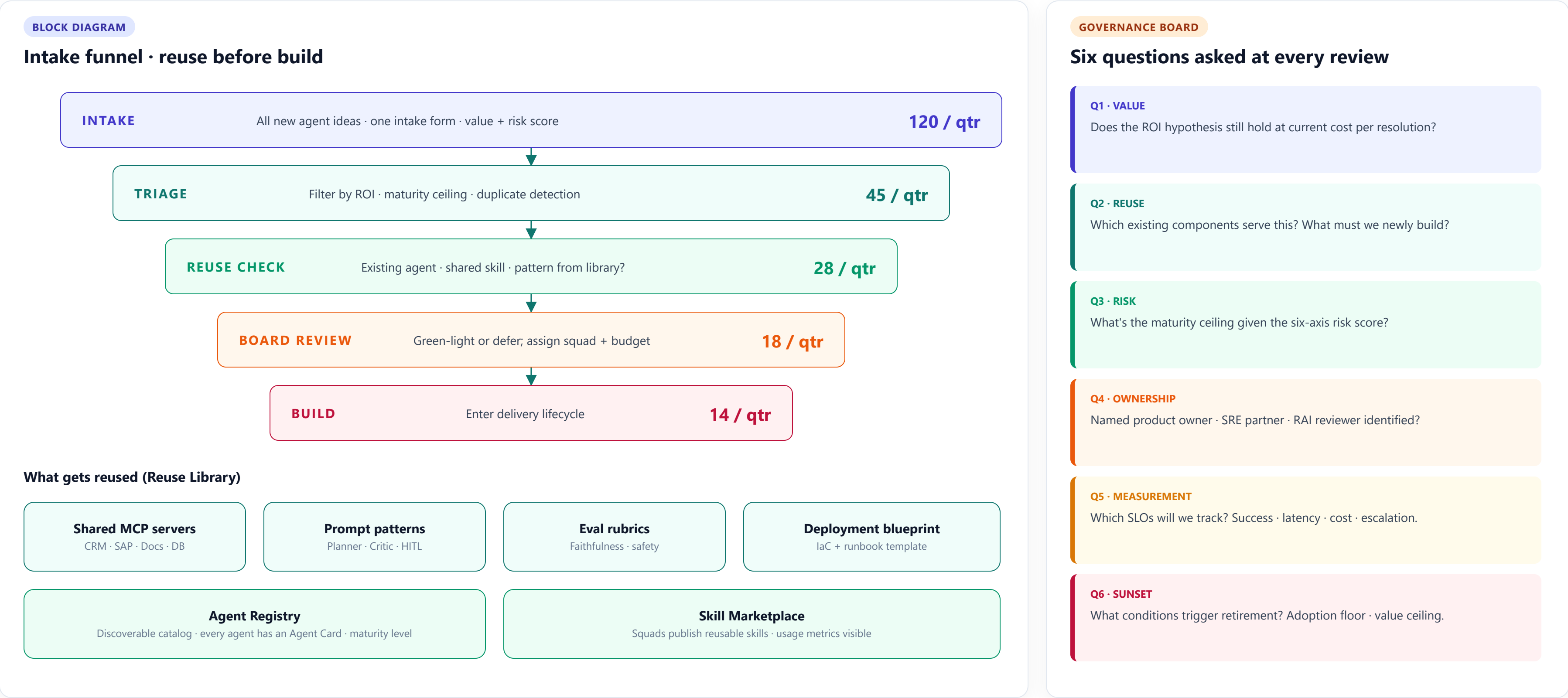
Live board · updated every stand-up

Type	Item	Severity	Owner	Mitigation / next step	Due
RISK	Model provider rate-limits during Q4 peak	HIGH	Platform team	Reserve PTU capacity · secondary region failover · queue-based throttle	Wave 3 exit
ASSUMPTION	CRM API stable through go-live (no schema change)	MED	CRM team lead	Contract-test suite in CI · change-notice from CRM team required 2 wks ahead	Ongoing
ISSUE	Groundedness score below target in French locale	MED	Squad · Support	Expand French embeddings · translate 500 golden cases · re-run eval	Week 4
DEPENDENCY	SSO integration with legacy IDP	MED	Identity team	Entra federation pattern agreed · migration ticket #4821 open	Wave 2 start
RISK	Prompt injection via user-uploaded PDFs	HIGH	RAI Office	Content-role tagging · sandbox tool args · red-team test suite added	Wave 1 exit

Tabletop cadence: weekly 45-min session with squad + Platform + RAI reps. Every open HIGH item gets a named owner and a date.

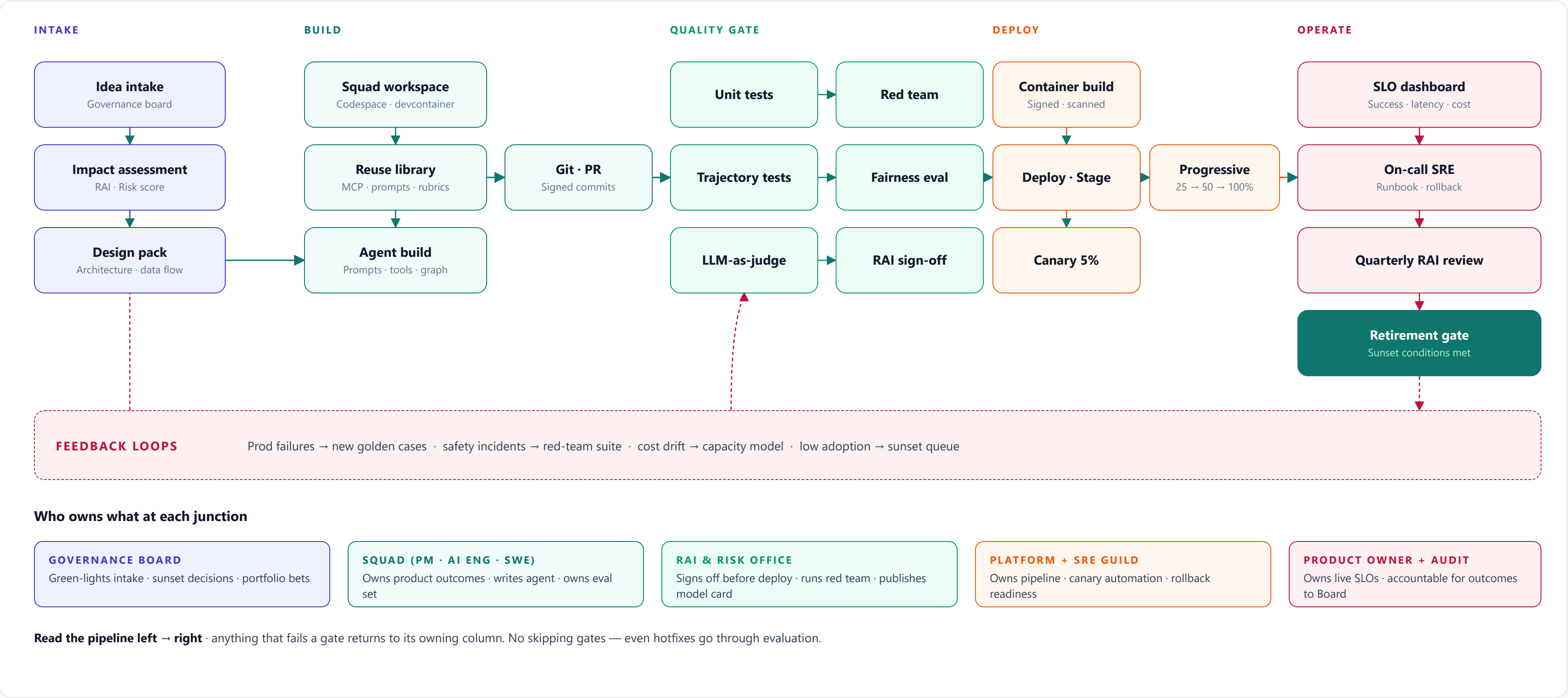
Portfolio Governance & Reuse

Managing a growing estate: intake, reuse of shared components, and the six-question governance board.



End-to-End Delivery Pipeline Wiring

A single wiring diagram that ties intake, build, gate, deploy, and operate — with the human roles that own each junction.



Portfolio Map — Where the Agents Sit

One picture of the full estate: agents by domain, their maturity, their reuse of shared platform components, and A2A links between them.

